

REPUBIQUE DU SENEGAL

Un peuple Un But Une Foi

Ministère de l'Enseignement Supérieur de la Recherche et de l'Innovation (MESRI)

UNIVERSITE ALIOUNE DIOP DE BAMBEY



UFR : SCIENCES APPLIQUÉES ET TECHNOLOGIES DE L'INFORMATION ET DE LA COMMUNICATION(SATIC)

FILIERE : SYSTÈMES ET RÉSEAUX (SR)

NIVEAU : MASTER 1

Rapport Automatisation et Déploiement d'une Infrastructure Réseau Virtualisé

Présenté par :

Mohamadou Sabaly

Professeur :

Dr. Babou

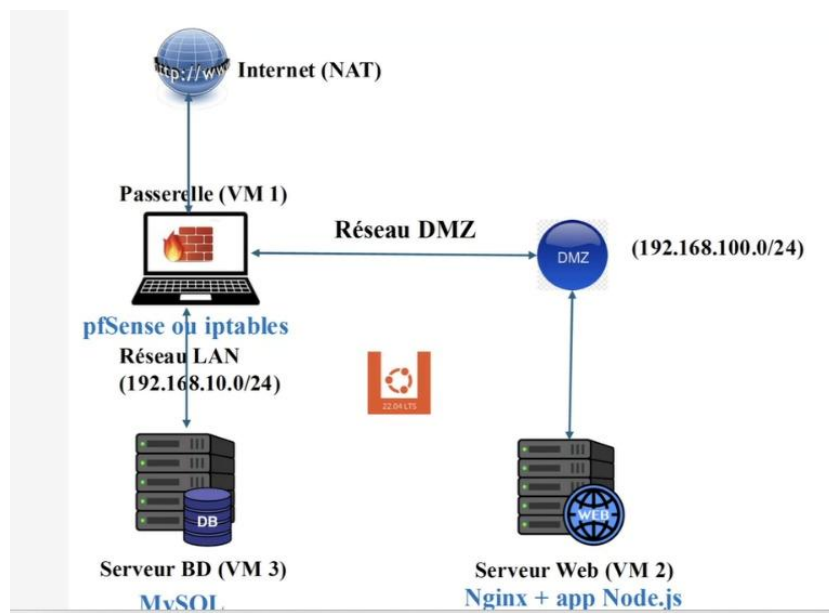
Année académique 2025-2026

Table des matières

Introduction	2
Architecture et Virtualisation (Vagrant)	2
La Passerelle (Routeur/Pare-feu).....	2
Le Serveur Web (DMZ) :	2
Le Serveur Base de Données (LAN) :.....	2
Gestion de la Configuration (Ansible)	4
Routage interne :	4
Installation des services.....	5
Sécurisation et Filtrage Réseau (Iptables)	6
Mise en place du NAT (Masquerade) :.....	6
Règles de Pare-feu (Filtrage) :	7
Preuve de concept (Crash-test) :.....	7
Automatisation DevOps et CI/CD (GitHub Actions).....	10
Configuration du Self-Hosted Runner :	11
Création du Workflow (deploy.yml) :	11
Déploiement applicatif automatisé :.....	12
Conclusion.....	14

Introduction

Dans le cadre de notre étude, nous avons conçu et déployé une architecture réseau virtualisée complète, simulant un environnement d'entreprise moderne. L'objectif de ce projet était de mettre en pratique les concepts **d'Infrastructure as Code (IaC)**, de sécurité réseau via le cloisonnement (**DMZ/LAN**), et d'automatiser les déploiements grâce à une approche DevOps (**CI/CD**). L'ensemble de l'infrastructure a été défini par du code, ce qui permet de la recréer, de la configurer et de la mettre à jour de manière entièrement automatisée.



Architecture et Virtualisation (Vagrant)

Pour construire notre environnement de test, nous avons utilisé l'outil **Vagrant** couplé à l'hyperviseur VirtualBox. L'utilisation d'un fichier **Vagrantfile** nous a permis de définir trois machines virtuelles sous Ubuntu de manière déclarative :

La Passerelle (Routeur/Pare-feu) : Le cœur du réseau. Elle dispose de trois interfaces réseau : une connectée en NAT (pour l'accès à Internet), une connectée au réseau DMZ (192.168.100.254), et une connectée au réseau privé LAN (192.168.10.254).

Le Serveur Web (DMZ) : Possède l'adresse IP 192.168.100.10. Cette machine a vocation à être exposée aux utilisateurs pour héberger notre site web.

Le Serveur Base de Données (LAN) : Possède l'adresse IP 192.168.10.10. Cette machine est strictement isolée dans le réseau privé pour protéger les données sensibles.

```

1 Vagrant.configure("2") do |config|
2   # Utilisation d'Ubuntu 22.04 pour toutes les machines
3   config.vm.box = "ubuntu/jammy64"
4
5   # =====
6   # VM 1 : Passerelle (Pare-feu / Routeur)
7   # =====
8   config.vm.define "passerelle" do |gw|
9     gw.vm.hostname = "passerelle"
10
11     gw.vm.network "private_network", ip: "192.168.100.254"
12     gw.vm.network "private_network", ip: "192.168.10.254"
13
14     gw.vm.provider "virtualbox" do |vb|
15       vb.memory = "1024"
16       vb.cpus = 1
17       vb.name = "Passerelle_VM1"
18       # Réparation DNS VirtualBox
19       vb.customize ["modifyvm", :id, "--natdnshostresolver1", "on"]
20       vb.customize ["modifyvm", :id, "--natdnspoxy1", "on"]
21     end
22
23     # === FORCER LE DNS AVANT ANSIBLE ===
24     gw.vm.provision "shell", inline: "rm -f /etc/resolv.conf && echo 'nameserver 8.8.8.8' > /etc/resolv.conf"
25
26     # === Configuration Ansible local ===
27     gw.vm.provision "ansible_local" do |ansible|
28       ansible.playbook = "ansible/passerelle.yml"
29     end
30   end
31
32   # =====
33   # VM 2 : Serveur Web (DMZ)
34   # =====
35   config.vm.define "web" do |web|
36     web.vm.hostname = "serveur-web"
37     web.vm.network "private_network", ip: "192.168.100.10"
38
39     web.vm.provider "virtualbox" do |vb|
40       vb.memory = "1024"
41       vb.cpus = 1
42       vb.name = "Serveur_Web_VM2"
43       # Réparation DNS VirtualBox
44       vb.customize ["modifyvm", :id, "--natdnshostresolver1", "on"]
45       vb.customize ["modifyvm", :id, "--natdnspoxy1", "on"]
46     end
47
48     # === FORCER LE DNS AVANT ANSIBLE ===
49     web.vm.provision "shell", inline: "rm -f /etc/resolv.conf && echo 'nameserver 8.8.8.8' > /etc/resolv.conf"
50
51     # === Provision Web ===
52     web.vm.provision "ansible_local" do |ansible|
53       ansible.playbook = "ansible/web.yml"
54     end
55   end
56
57   # =====
58   # VM 3 : Serveur de Base de Données (LAN)
59   # =====
60   config.vm.define "db" do |db|
61     db.vm.hostname = "serveur-bd"
62     db.vm.network "private_network", ip: "192.168.10.10"
63
64     db.vm.provider "virtualbox" do |vb|
65       vb.memory = "1024"
66       vb.cpus = 1
67       vb.name = "Serveur_BD_VM3"
68       # Réparation DNS VirtualBox
69       vb.customize ["modifyvm", :id, "--natdnshostresolver1", "on"]
70       vb.customize ["modifyvm", :id, "--natdnspoxy1", "on"]
71     end
72
73     # === FORCER LE DNS AVANT ANSIBLE ===
74     db.vm.provision "shell", inline: "rm -f /etc/resolv.conf && echo 'nameserver 8.8.8.8' > /etc/resolv.conf"
75
76     # === Provision DB ===
77     db.vm.provision "ansible_local" do |ansible|
78       ansible.playbook = "ansible/db.yml"
79     end
80   end
81
82 end

```

Gestion de la Configuration (Ansible)

Afin d'éviter toute configuration manuelle et de garantir la reproductibilité de nos serveurs, nous avons utilisé **Ansible**. Dès la création des machines par **Vagrant**, **Ansible** prend le relais (**Provisioning**) à travers différents "playbooks" (fichiers **YAML**) :

Routage interne : Nous avons configuré Ansible pour modifier la table de routage des serveurs **Web** et **DB**, forçant leur trafic sortant à passer par notre machine Passerelle.

➤ Code *db.yml*

```
1  ---
2  - name: Configuration du Serveur de Base de Données (LAN)
3    hosts: all
4    become: yes
5    tasks:
6      - name: Forcer la route par défaut vers la Passerelle (VM 1)
7        command: ip route replace default via 192.168.10.254
8
9      - name: Mettre à jour le cache APT
10     apt:
11       update_cache: yes
12
13     - name: Installer MySQL Server
14       apt:
15         name: mysql-server
16         state: present
```

➤ Code *web.yml*

```
1 ---
2 - name: Configuration du Serveur Web (DMZ)
3   hosts: all
4   become: yes
5   tasks:
6
7     - name: Forcer la route par défaut vers la Passerelle (VM 1)
8       command: ip route replace default via 192.168.100.254
9
10    - name: Mettre à jour le cache APT
11      apt:
12        update_cache: yes
13
14    - name: Installer Nginx
15      apt:
16        name: nginx
17        state: present
18
19    # =====
20    # NOUVEAU : DÉPLOIEMENT DU SITE WEB
21    # =====
22
23    - name: Installer Git
24      apt:
25        name: git
26        state: present
27
28    - name: Supprimer la page par défaut de Nginx pour faire de la place
29      file:
30        path: /var/www/html/index.nginx-debian.html
31        state: absent
32
33    - name: Cloner le site Java P00 depuis GitHub vers le dossier Nginx
34      git:
35        repo: 'https://github.com/hamousabs95/javaP00.git'
36        dest: /var/www/html/
37        force: yes
```

Installation des services : Ansible a été chargé de mettre à jour les paquets APT, d'installer **Nginx** et **Git** sur le serveur **Web**, ainsi que d'installer le service **MySQL** sur le serveur de Base de données.

```

18
19 # =====
20 # NOUVEAU : DÉPLOIEMENT DU SITE WEB
21 # =====
22
23 - name: Installer Git
24   apt:
25     name: git
26     state: present
27
28 - name: Supprimer la page par défaut de Nginx pour faire de la place
29   file:
30     path: /var/www/html/index.nginx-debian.html
31     state: absent
32
33 - name: Cloner le site Java P00 depuis GitHub vers le dossier Nginx
34   git:
35     repo: 'https://github.com/hamousabs95/javaP00.git'
36     dest: /var/www/html/
37     force: yes

```

Sécurisation et Filtrage Réseau (Iptables)

Une infrastructure d'entreprise nécessite une sécurité rigoureuse. Nous avons donc transformé notre machine "**Passerelle**" en un véritable routeur et pare-feu en utilisant **Iptables** (automatisé via Ansible) :

Mise en place du NAT (Masquerade) : Nous avons configuré le NAT pour permettre aux machines des réseaux internes (**Web** et **DB**) d'accéder à Internet (indispensable pour télécharger les mises à jour), sans pour autant être exposées directement.

```

1 ---
2 - name: Configuration de la Passerelle (Routeur, NAT & Pare-feu)
3   hosts: all
4   become: yes
5   tasks:
6     - name: Activer le routage IP (IP Forwarding)
7       sysctl:
8         name: net.ipv4.ip_forward
9         value: '1'
10        sysctl_set: yes
11        state: present
12        reload: yes
13
14    - name: Installer iptables-persistent pour sauvegarder les règles
15      apt:
16        name: iptables-persistent
17        state: present
18        update_cache: yes
19
20    - name: Configurer le NAT pour donner accès à Internet (enp0s3)
21      iptables:
22        table: nat
23        chain: POSTROUTING
24        out_interface: enp0s3
25        jump: MASQUERADE
26

```

Règles de Pare-feu (Filtrage) : La règle d'or de notre architecture est que la zone exposée (**DMZ**) ne doit pas pouvoir compromettre la zone sécurisée (**LAN**). Nous avons donc écrit des règles strictes :

```
27 # =====
28 # NOUVEAU : RÈGLES DE SÉCURITÉ (PARE-FEU)
29 # =====
30
31 - name: 1. Autoriser le trafic de retour (connexions déjà établies)
32   iptables:
33     chain: FORWARD
34     ctstate: ESTABLISHED,RELATED
35     jump: ACCEPT
36
37 - name: 2. Autoriser le ping entre tous les réseaux (pour nos tests)
38   iptables:
39     chain: FORWARD
40     protocol: icmp
41     jump: ACCEPT
42
43 - name: 3. Autoriser le Web (DMZ) à parler à la DB (LAN) UNIQUEMENT sur MySQL (Port 3306)
44   iptables:
45     chain: FORWARD
46     source: 192.168.100.10
47     destination: 192.168.10.10
48     protocol: tcp
49     destination_port: 3306
50     jump: ACCEPT
51
52 - name: 4. BLOQUER tout le reste du trafic de la DMZ vers le LAN (Protection anti-piratage)
53   iptables:
54     chain: FORWARD
55     source: 192.168.100.0/24
56     destination: 192.168.10.0/24
57     jump: REJECT
58
59 # =====
60
61 - name: Sauvegarder toutes les règles iptables
62   command: netfilter-persistent save
```

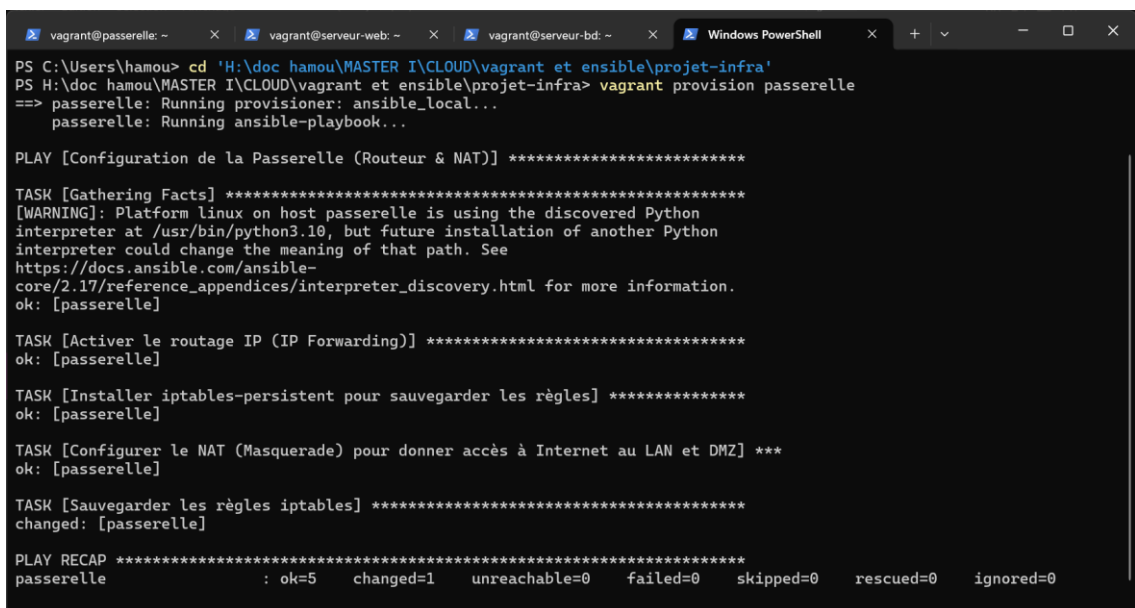
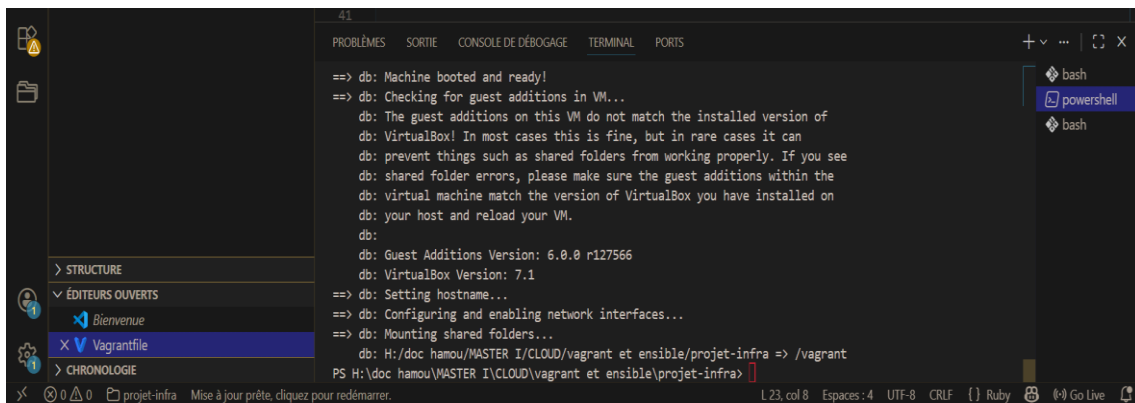
- Autorisation du protocole **ICMP** (Ping) pour faciliter nos tests de diagnostic réseau.
- Autorisation du trafic depuis le serveur Web vers la Base de données uniquement sur le port **TCP 3306** (nécessaire au fonctionnement de l'application).
- Blocage total (**REJECT**) de tout le reste du trafic provenant de la **DMZ** vers le **LAN**.

Preuve de concept (Crash-test) :

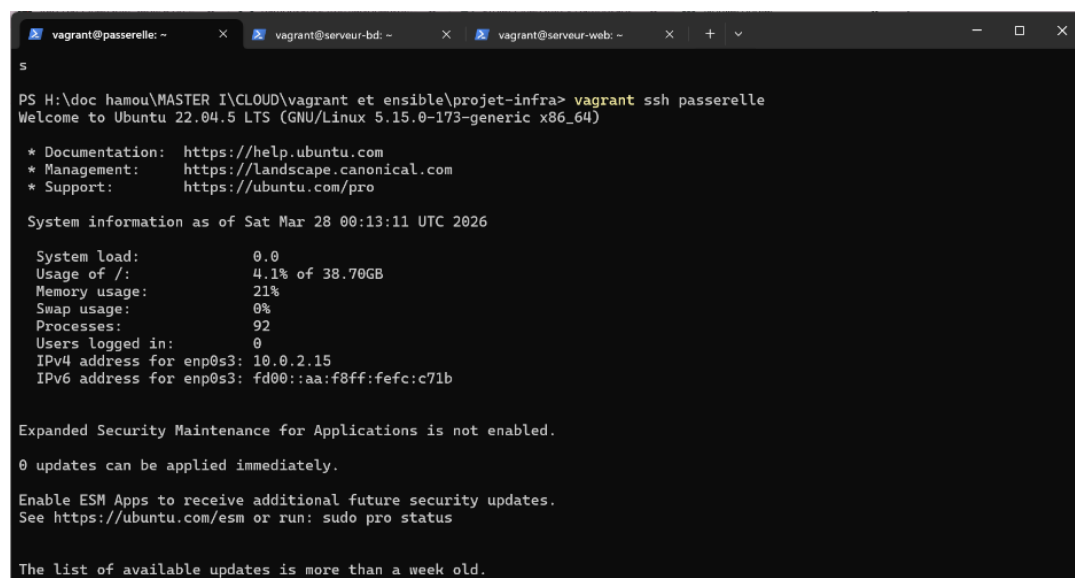
NB:

Avant de passer au teste nous avons démarre le service avec la commande *vagrant up*

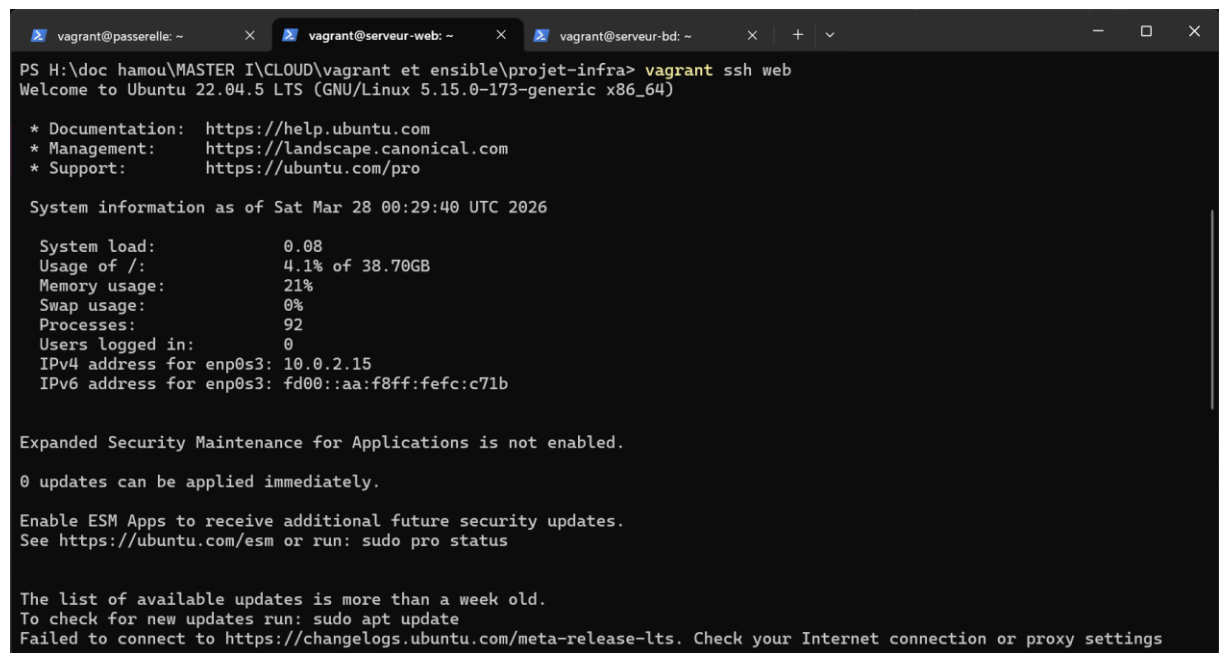
Et provisionner les règle (ansible)par la commande *vagrant provisioning*



➤ **Interface Machin passerelle** (on se connecte avec la commande *vagrant ssh* *Passarelle*)



➤ **Interface Machin web** (on se connecte avec la commande *vagrant ssh web*)



```
vagrant@passerelle: ~
vagrant@serveur-web: ~
vagrant@serveur-bd: ~
PS H:\doc hamou\MASTER I\CLOUD\vagrant et ensible\projet-infra> vagrant ssh web
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-173-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sat Mar 28 00:29:40 UTC 2026

System load:          0.08
Usage of /:           4.1% of 38.70GB
Memory usage:         21%
Swap usage:           0%
Processes:            92
Users logged in:      0
IPv4 address for enp0s3: 10.0.2.15
IPv6 address for enp0s3: fd00::aa:f8ff:fefc:c71b

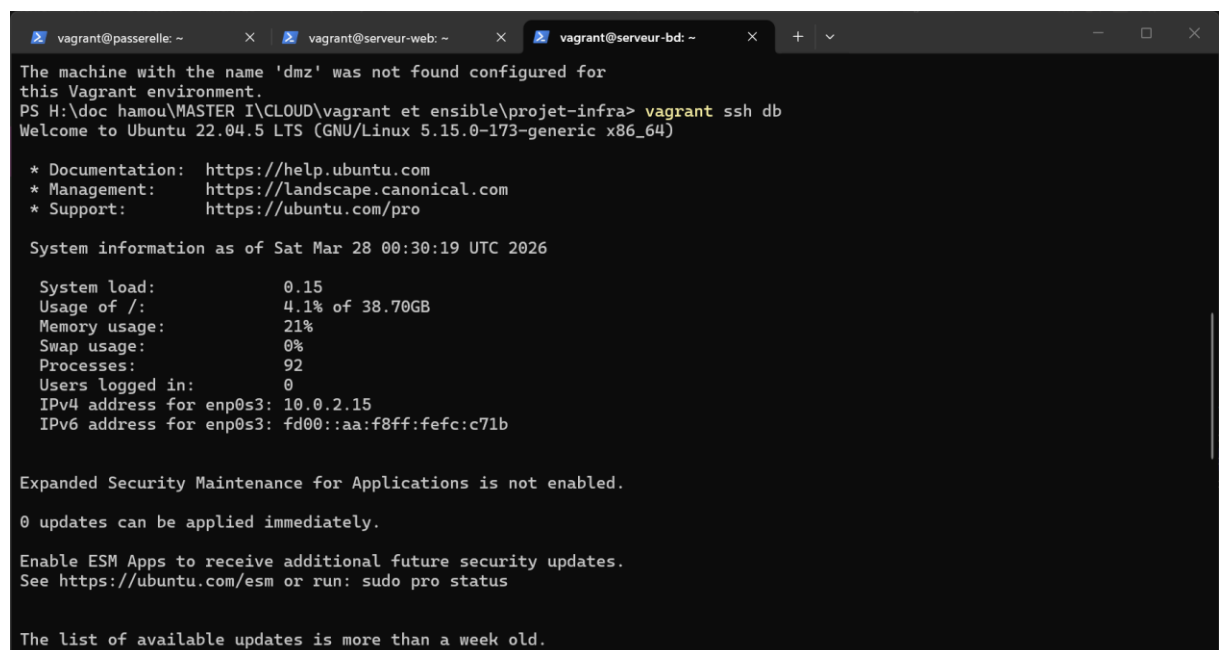
Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection or proxy settings
```

➤ **Interface Machin bd** (on se connecte avec la commande *vagrant ssh bd*)



```
vagrant@passerelle: ~
vagrant@serveur-web: ~
vagrant@serveur-bd: ~
The machine with the name 'dmz' was not found configured for
this Vagrant environment.
PS H:\doc hamou\MASTER I\CLOUD\vagrant et ensible\projet-infra> vagrant ssh db
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-173-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sat Mar 28 00:30:19 UTC 2026

System load:          0.15
Usage of /:           4.1% of 38.70GB
Memory usage:         21%
Swap usage:           0%
Processes:            92
Users logged in:      0
IPv4 address for enp0s3: 10.0.2.15
IPv6 address for enp0s3: fd00::aa:f8ff:fefc:c71b

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
```

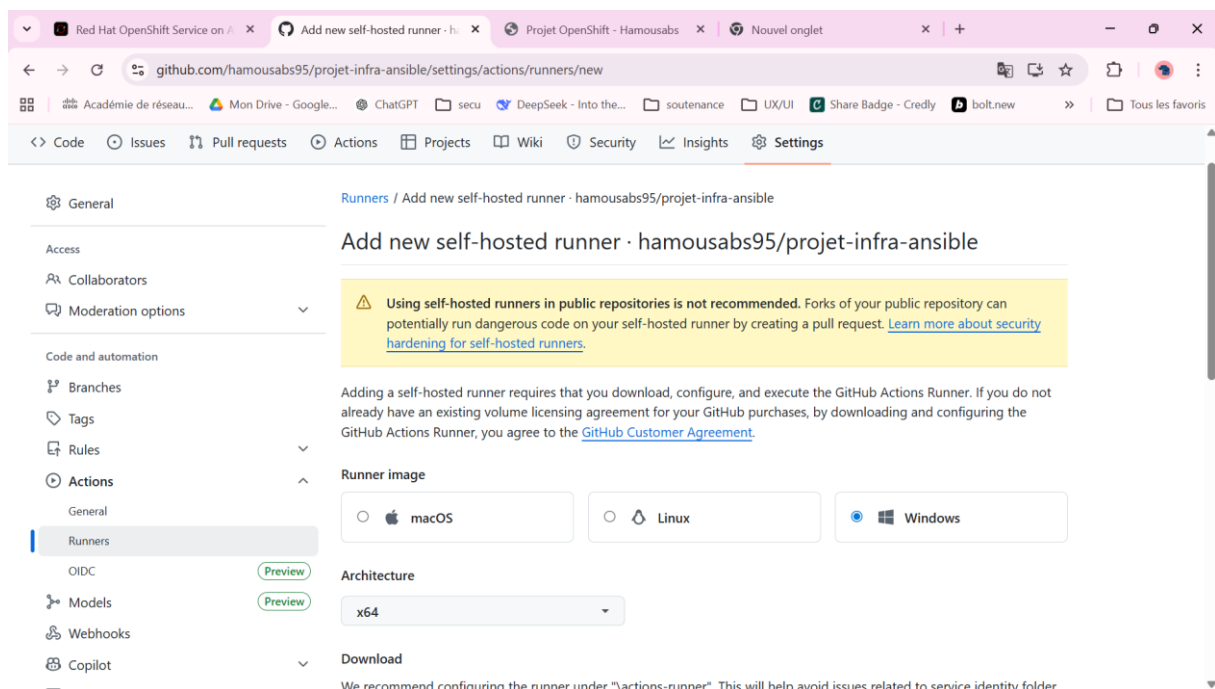
Comme le démontre la capture ci-dessous, le ping depuis le serveur Web vers la Base de données fonctionne parfaitement (**0% packet loss**), prouvant que le routage est opérationnel. Cependant, une tentative de connexion SSH (port 22) depuis le serveur Web vers la Base de données est immédiatement bloquée par notre pare-feu (**Connection refused**). Le cloisonnement est donc un succès.

```
vagrant@serveur-web:~$ ping 192.168.10.10
PING 192.168.10.10 (192.168.10.10) 56(84) bytes of data.
64 bytes from 192.168.10.10: icmp_seq=1 ttl=63 time=1.92 ms
64 bytes from 192.168.10.10: icmp_seq=2 ttl=63 time=8.02 ms
^C
--- 192.168.10.10 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 1.923/4.970/8.017/3.047 ms
vagrant@serveur-web:~$ ping -c 3 192.168.10.10
PING 192.168.10.10 (192.168.10.10) 56(84) bytes of data.
64 bytes from 192.168.10.10: icmp_seq=1 ttl=63 time=2.97 ms
64 bytes from 192.168.10.10: icmp_seq=2 ttl=63 time=28.1 ms
64 bytes from 192.168.10.10: icmp_seq=3 ttl=63 time=2.54 ms

--- 192.168.10.10 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 2.543/11.214/28.135/11.965 ms
vagrant@serveur-web:~$ ssh 192.168.10.10
ssh: connect to host 192.168.10.10 port 22: Connection refused
vagrant@serveur-web:~$
```

Automatisation DevOps et CI/CD (GitHub Actions)

Pour couronner ce projet, nous avons implémenté un pipeline de déploiement continu (CI/CD) permettant de lier notre code source stocké sur GitHub à notre infrastructure locale.



Configuration du Self-Hosted Runner : GitHub étant hébergé dans le Cloud, il ne pouvait pas communiquer directement avec nos machines virtuelles locales. Nous avons donc installé et configuré un Runner **auto-hébergé** sur notre machine hôte **Windows**. Ce Runner écoute les événements de notre dépôt GitHub.

```
Windows PowerShell
PS C:\Users\hamou\OneDrive\Documents\runner\actions-runner> Invoke-WebRequest -Uri https://github.com/actions/runner/releases/download/v2.333.0/actions-runner-win-x64-2.333.0.zip -OutFile actions-runner-win-x64-2.333.0.zip
PS C:\Users\hamou\OneDrive\Documents\runner\actions-runner> if((Get-FileHash -Path actions-runner-win-x64-2.333.0.zip -Algorithm SHA256).Hash.ToUpper() -ne '7176d0c4b674d4108b515503a53b4bc9eeab9339c645e274a97c142fe1c64b95'.ToUpper()){ throw 'Computed checksum did not match' }
PS C:\Users\hamou\OneDrive\Documents\runner\actions-runner> Add-Type -AssemblyName System.IO.Compression.FileSystem ; [System.IO.Compression.ZipFile]::ExtractToDirectory("$PWD/actions-runner-win-x64-2.333.0.zip", "$PWD")
PS C:\Users\hamou\OneDrive\Documents\runner\actions-runner> ./config.cmd --url https://github.com/hamousabs95/projet-inf-ra-ansible --token BFHILUKPROBMD7F3QM4ZVYDJY5LM4

-----
  G I T H U B  A c t i o n s
-----
Self-hosted runner registration

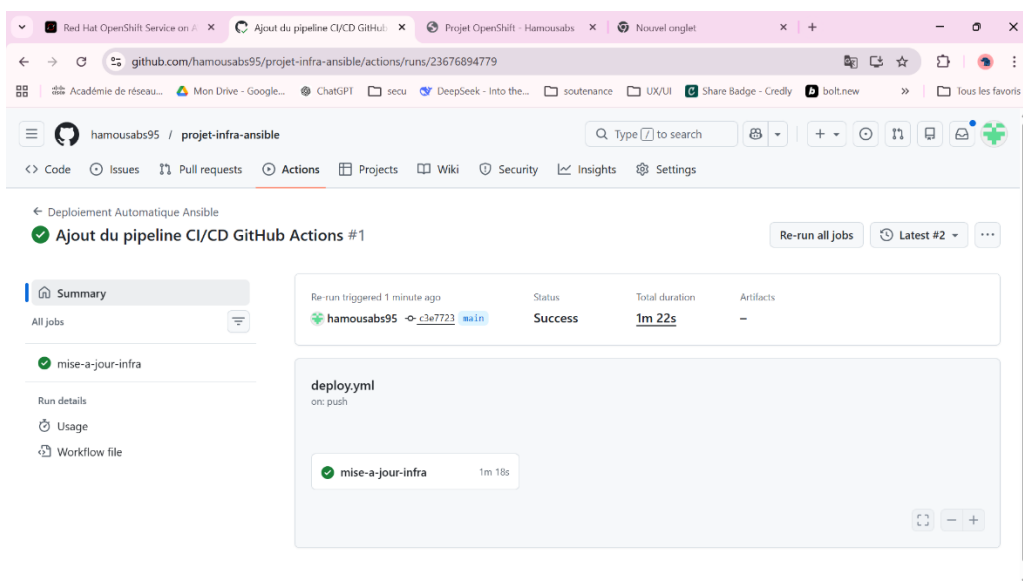
# Authentication

V Connected to GitHub

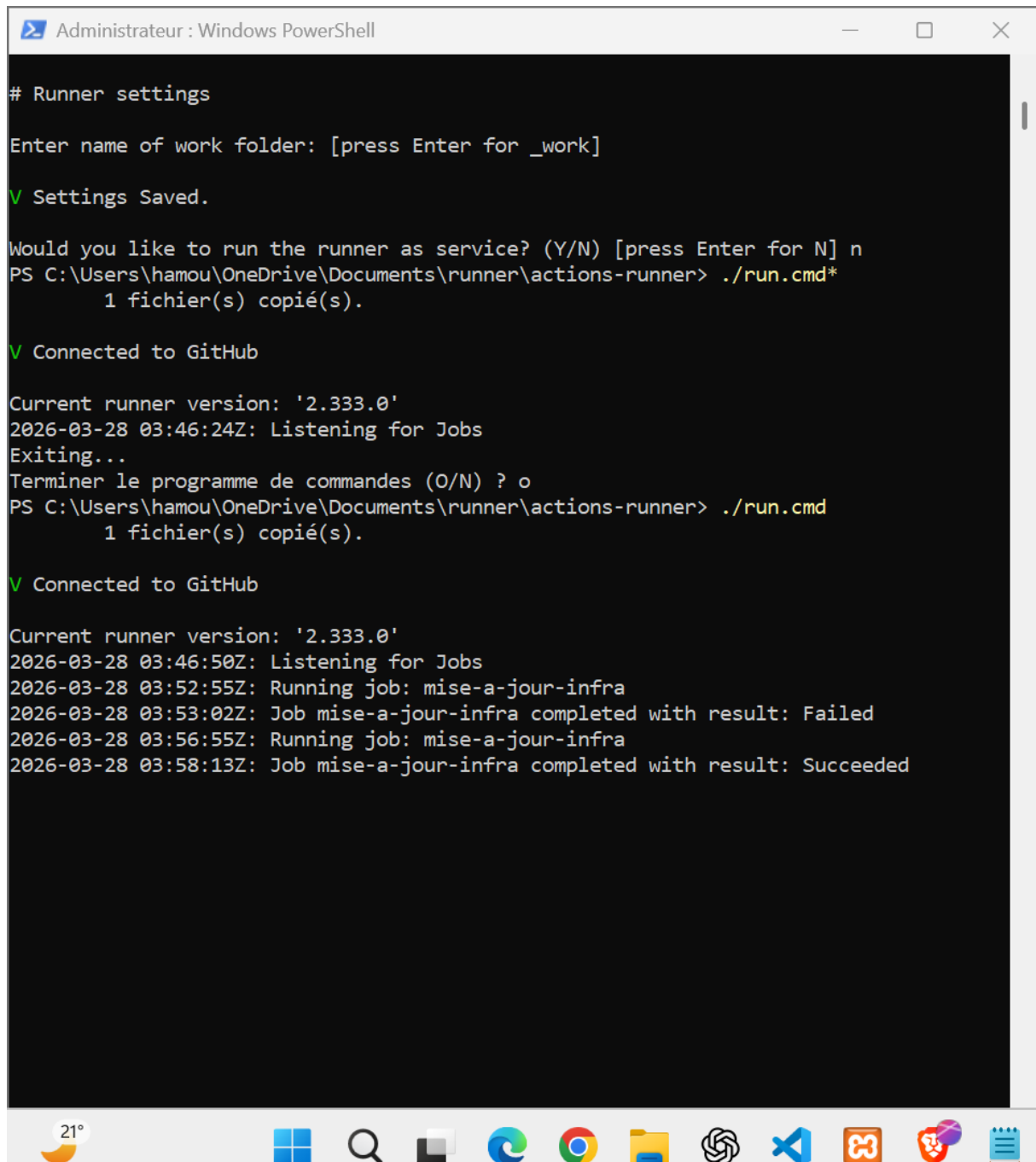
# Runner Registration

Enter the name of the runner group to add this runner to: [press Enter for Default] |
```

Création du Workflow (deploy.yml) : Nous avons rédigé un pipeline qui se déclenche automatiquement à chaque git push sur la branche principale. Ce pipeline ordonne au Runner d'exécuter la commande vagrant provision.



Déploiement applicatif automatisé : Grâce à cette automatisation, une simple mise à jour de notre code déclenche **Ansible**, qui va lui-même cloner la dernière version de notre projet de développement ("**Java POO**") depuis GitHub et l'injecter directement dans le dossier public de **Nginx** (*/var/www/html*).



```
# Runner settings

Enter name of work folder: [press Enter for _work]

V Settings Saved.

Would you like to run the runner as service? (Y/N) [press Enter for N] n
PS C:\Users\hamou\OneDrive\Documents\runner\actions-runner> ./run.cmd*
    1 fichier(s) copié(s).

V Connected to GitHub

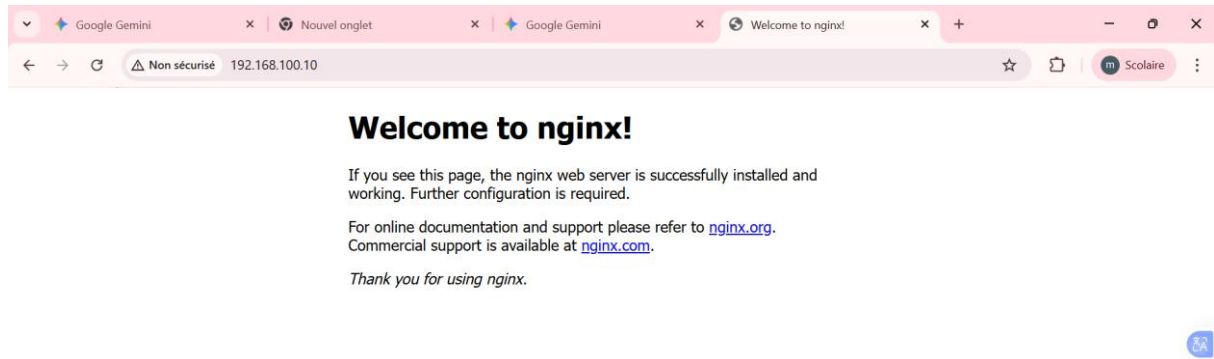
Current runner version: '2.333.0'
2026-03-28 03:46:24Z: Listening for Jobs
Exiting...
Terminer le programme de commandes (O/N) ? o
PS C:\Users\hamou\OneDrive\Documents\runner\actions-runner> ./run.cmd
    1 fichier(s) copié(s).

V Connected to GitHub

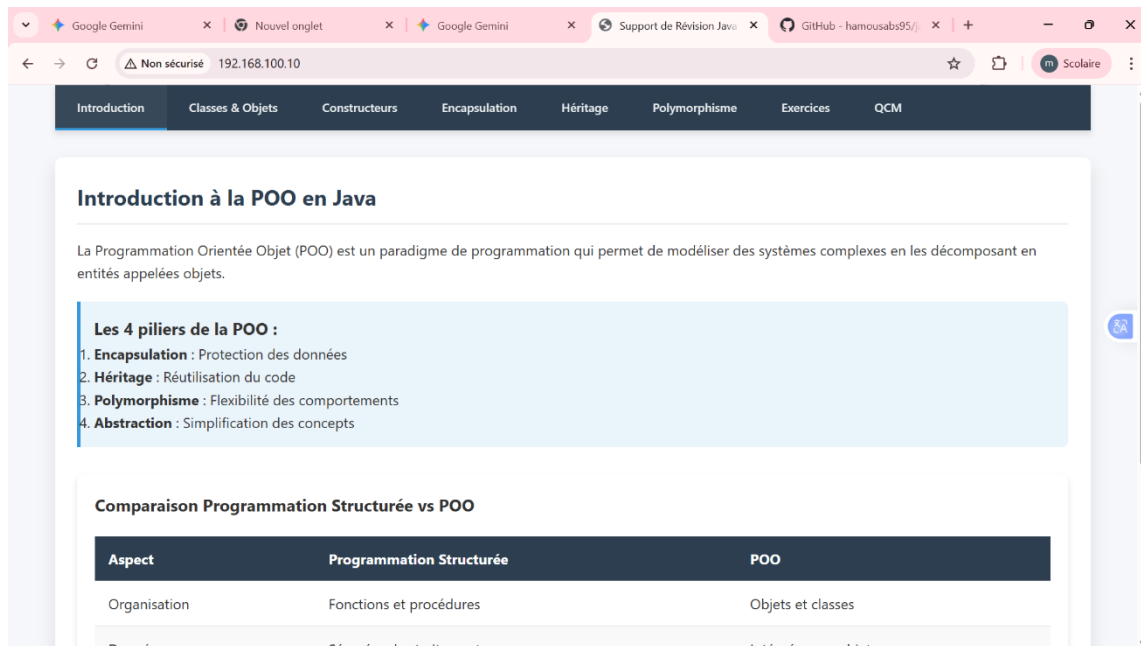
Current runner version: '2.333.0'
2026-03-28 03:46:50Z: Listening for Jobs
2026-03-28 03:52:55Z: Running job: mise-a-jour-infra
2026-03-28 03:53:02Z: Job mise-a-jour-infra completed with result: Failed
2026-03-28 03:56:55Z: Running job: mise-a-jour-infra
2026-03-28 03:58:13Z: Job mise-a-jour-infra completed with result: Succeeded
```

Le processus est entièrement transparent et autonome : le site web est mis en ligne et mis à jour sans aucune intervention manuelle sur les serveurs.

➤ Notre serveur web avant le déploiement sur GitHub



➤ Notre serveur web Après le déploiement sur GitHub



Conclusion

Ce projet nous a permis de valider des compétences clés en administration système, en réseau et en culture DevOps. En partant de simples fichiers de code, nous sommes capables de déployer une architecture réseau complexe, d'en assurer la sécurité par un cloisonnement strict des flux, et de mettre en œuvre un cycle de déploiement logiciel continu. L'environnement créé est à la fois robuste, reproductible et prêt pour la production.